Lógica y Matemática Discreta — Guía de Estudio

17 tipos de ejercicio con pasos, fórmulas y particularidades.

TEMA 1 — Lógica Proposicional

Tipo 1 — Tablas de verdad y clasificación

Construir la tabla de verdad de una fórmula y clasificarla.

☑ ALGORITMO
1. Contar variables (p, q, r...) → número de filas = 2^n
2. Crear columnas: variables → subfórmulas → fórmula completa
3. Rellenar de dentro hacia fuera usando las reglas de cada conector
4. Clasificar: todas V → TAUTOLOGÍA | todas F → CONTRADICCIÓN | mezcla → CONTINGENTE

CONECTORES — reglas clave
¬p: niega el valor (V→F, F→V)
p ∧ q: V solo cuando los dos son V
p ∨ q: F solo cuando los dos son F
p → q: F SOLO cuando p=V y q=F (todos los demás casos son V)
p ↔ q: V cuando los dos son IGUALES (ambos V o ambos F)

⚠ TRAMPAS FRECUENTES
La implicación → solo falla en UN caso: V→F. Todo lo demás es V.
El bicondicional ↔ es V cuando son IGUALES, no cuando son verdaderos.
Para ser tautología TODAS las filas deben ser V, no solo la mayoría.

Tipo 2 — Leyes de De Morgan

Simplificar negaciones de conjunciones y disyunciones.

☑ REGLAS DE DE MORGAN
¬(p ∧ q) = ¬p ∨ ¬q (la negación entra y cambia ∧ por ∨)
¬(p ∨ q) = ¬p ∧ ¬q (la negación entra y cambia ∨ por ∧)
¬¬p = p (doble negación se cancela)
Equivalencia clave: p → q = ¬p ∨ q
Negación de implicación: ¬(p → q) = p ∧ ¬q

⚠ TRAMPAS FRECUENTES
¬(p ∧ ¬q): la doble negación sobre q da ¬(¬q) = q, no ¬q.
Siempre aplicar doble negación después de De Morgan.

Tipo 3 — Circuitos lógicos

Obtener la función booleana de un circuito lógico.

☑ ALGORITMO
1. Leer el circuito de IZQUIERDA a DERECHA, capa por capa
2. Nombrar la salida de cada puerta con una letra (A, B, C...)
3. Ir combinando las salidas hasta obtener la función final

PUERTAS LÓGICAS
AND (∧): forma D achatada → x ∧ y
OR (∨): forma D con curva → x ∨ y
NOT (¬): triángulo con círculo → ¬x
NAND: AND + círculo en salida → ¬(x ∧ y)
NOR: OR + círculo en salida → ¬(x ∨ y)
XOR (⊕): forma OR con curva extra → x ⊕ y = V solo cuando son DISTINTOS

TEMA 2 — Lógica de Predicados

Tipo 4 — Formalizar enunciados

Traducir frases en lenguaje natural a símbolos lógicos.

CUANTIFICADORES
∀x → 'para todo x' (tiene que cumplirse para TODOS)
∃x → 'existe algún x' (basta con que se cumpla para UNO)
El orden importa: ∀x ∃y ≠ ∃y ∀x (NO conmutan)

MÉTODO
1. Identificar si es 'para todo' (∀) o 'existe' (∃)
2. Identificar el universo (ℕ, ℤ, ℝ...)
3. Identificar el predicado (la condición)
4. Respetar el orden de los cuantificadores

TRAMPAS FRECUENTES
'Para todo x, existe y' ≠ 'Existe y, para todo x': el orden cambia el significado completamente.
'Existe x par e impar': un SOLO número cumple las dos → ∃x: p(x) ∧ q(x)
'Existe x par Y existe x impar': dos números distintos → [∃x: p(x)] ∧ [∃y: q(y)]
Doble negación: 'no es cierto que sea falso' = ¬¬ = se cancela.

Tipo 5 — Comparar enunciados con cuantificadores

Determinar si dos enunciados parecidos tienen el mismo significado.

MÉTODO
1. Formalizar cada enunciado por separado
2. Comprobar si el orden de los cuantificadores es el mismo
3. Determinar si son verdaderos o falsos (buscar contraejemplos)

⚠ TRAMPAS FRECUENTES
$\forall x\,[p(x) \vee q(x)] \neq [\forall x\, p(x)] \vee [\forall x\, q(x)]$ (el ∀ NO se distribuye sobre ∨)
$\forall x\,[p(x) \wedge q(x)] = [\forall x\, p(x)] \wedge [\forall x\, q(x)]$ (el ∀ SÍ se distribuye sobre ∧)
$\exists x\,[p(x) \vee q(x)] = [\exists x\, p(x)] \vee [\exists x\, q(x)]$ (el ∃ SÍ se distribuye sobre ∨)
$\exists x\,[p(x) \wedge q(x)] \neq [\exists x\, p(x)] \wedge [\exists x\, q(x)]$ (el ∃ NO se distribuye sobre ∧)

Tipo 6 — Simplificar negaciones de cuantificadores

Empujar la negación hacia dentro hasta llegar al predicado.

☑ REGLAS — aplicar en orden de izquierda a derecha	
$\neg\forall x \rightarrow \exists x : \neg(...)$	(negar 'para todo' da 'existe que no')
$\neg\exists x \rightarrow \forall x, \neg(...)$	(negar 'existe' da 'para todo que no')
$\neg(p \rightarrow q) \rightarrow p \wedge \neg q$	(negación de implicación)
$\neg(p \wedge q) \rightarrow \neg p \vee \neg q$	(De Morgan)
$\neg(p \vee q) \rightarrow \neg p \wedge \neg q$	(De Morgan)
$\neg\neg p \rightarrow p$	(doble negación)

📋 EJEMPLO COMPLETO
$\neg[\exists x : \forall y, p(x) \rightarrow q(x,y)]$
Paso 1 ($\neg\exists \rightarrow \forall$): $\forall x : \neg[\forall y, p(x) \rightarrow q(x,y)]$
Paso 2 ($\neg\forall \rightarrow \exists$): $\forall x : \exists y, \neg[p(x) \rightarrow q(x,y)]$
Paso 3 ($\neg(p\rightarrow q) = p\wedge\neg q$): $\forall x : \exists y, p(x) \wedge \neg q(x,y)$

Tipo 7 — Sustituciones

Aplicar $\sigma = (x_1|t_1; x_2|t_2; ...)$ a una fórmula α.

☑ ALGORITMO (ENCADENADO — según tu profesora)
1. Tomar la fórmula original α
2. Aplicar la PRIMERA sustitución ($x_1|t_1$): reemplazar x_1 en toda la fórmula

3. Aplicar la SEGUNDA sustitución ($x_2|t_2$) sobre el RESULTADO anterior

4. Repetir para cada sustitución EN ORDEN hasta terminar

EJEMPLO

α = p(x, f(y), g(x,y)) σ = (x|f(z); y|x)

Paso 1 (x→f(z)): p(f(z), f(y), g(f(z),y))

Paso 2 (y→x): p(f(z), f(x), g(f(z),x))

⚠ REGLAS IMPORTANTES

Solo las VARIABLES pueden sustituirse. Las funciones y constantes NO.

La notación es (variable|término), nunca al revés.

Las variables que no aparecen en σ se quedan igual.

Tipo 8 — Unificación

Determinar si dos fórmulas pueden igualarse mediante una sustitución.

ALGORITMO — paso a paso

INICIO: W_0 = {fórmulas a unificar}, σ_0 = () (sustitución vacía)

REPETIR mientras las fórmulas de W sean diferentes:

PASO 1 — Calcular Dk: comparar las fórmulas argumento por argumento

Dk = los primeros elementos en los que difieren

Ejemplo: p(a, f(x)) vs p(a, g(y)) → D = {f(x), g(y)}

PASO 2 — Comprobar si es posible unificar:

Si en Dk NO hay ninguna variable → PARAR: NO UNIFICABLE

Si hay variable pero aparece DENTRO del otro término → PARAR: NO UNIFICABLE

(ej: D={x, f(x)} → x aparece dentro de f(x) → circular → NO UNIFICABLE)

PASO 3 — Crear τk: elegir la variable vk y el término tk de Dk

τk = (vk|tk) → 'sustituir vk por tk'

PASO 4 — Actualizar:

σk+1 = (σk; τk) → acumular la sustitución

Wk+1 = τk(Wk) → aplicar τk sobre las fórmulas actuales

FIN: Si todas las fórmulas son iguales → UNIFICABLE, unificador = σ final

⚠ CASOS DE PARADA — NO UNIFICABLE

1. Dk tiene dos funciones distintas: ej. {f(x), g(x)} — ninguna es variable
2. Dk tiene dos constantes distintas: ej. {a, b}
3. La variable aparece dentro del término: ej. {x, f(x)} — circular

EJEMPLO

W = {p(a,x,f(g(y))), p(z,f(z),f(u))}, a constante

[k=0] D0={a,z} → τ0=(z|a) → W1={p(a,x,f(g(y))), p(a,f(a),f(u))}

[k=1] D1={x,f(a)} → τ1=(x|f(a)) → W2={p(a,f(a),f(g(y))), p(a,f(a),f(u))}

[k=2] D2={g(y),u} → τ2=(u|g(y)) → W3={p(a,f(a),f(g(y)))}

|W3|=1 → UNIFICABLE σ = (z|a; x|f(a); u|g(y))

TEMA 3 — Recurrencia e Inducción

Tipo 9 — Recurrencias Tipo 1: una sola raíz

La recurrencia usa solo x_{n-1} (un término anterior).

☑ ALGORITMO
1. RAÍZ: es el número que multiplica a x_{n-1}
Ejemplo: $x_n = 3 \cdot x_{n-1} \rightarrow r = 3$
2. SOLUCIÓN GENERAL: $x_n = a \cdot r^n$
3. ENCONTRAR a: sustituir n=0 → $a \cdot r^0 = a = x_0$
Es decir: $a = x_0$ directamente
4. RESULTADO: escribir x_n con el valor de a

📋 EJEMPLO
$x_0 = 4$, $x_n = 3 \cdot x_{n-1}$
Raíz: r = 3
Solución general: $x_n = a \cdot 3^n$
$a = x_0 = 4$
Resultado: $x_n = 4 \cdot 3^n$

Tipo 10 — Recurrencias Tipo 2: dos raíces distintas

La recurrencia usa x_{n-1} y x_{n-2} (dos términos anteriores), con discriminante positivo.

☑ ALGORITMO
1. POLINOMIO CARACTERÍSTICO: reescribir $x_n - a \cdot x_{n-1} - b \cdot x_{n-2} = 0$
Sustituir $x_n \rightarrow r^n$ y dividir entre $r^{n-2} \rightarrow r^2 - a \cdot r - b = 0$
2. RAÍCES: usar fórmula $r = (-b \pm \sqrt{(b^2-4ac)}) / 2a$
Si discriminante > 0 → dos raíces distintas r1 y r2
3. SOLUCIÓN GENERAL: $x_n = a \cdot r1^n + b \cdot r2^n$
4. SISTEMA: sustituir n=0 y n=1

n=0: $a + b = x_0$

n=1: $a \cdot r1 + b \cdot r2 = x_1$

5. Resolver el sistema y escribir xn con los valores de a y b

EJEMPLO

$x_0 = 0$, $x_1 = 6$, $xn = xn-1 + 2 \cdot xn-2$

Polinomio: $r^2 - r - 2 = 0$

Raíces: $r1 = 2$, $r2 = -1$

Solución general: $xn = a \cdot 2^n + b \cdot (-1)^n$

Sistema: $a+b=0$ y $2a-b=6 \rightarrow a=2, b=-2$

Resultado: $xn = 2 \cdot 2^n - 2 \cdot (-1)^n$

⚠ TRAMPAS FRECUENTES

El polinomio es de grado 2 porque usa DOS términos anteriores.

Si discriminante = 0 → raíz doble (ver Tipo 11).

Al resolver el sistema, sumar/restar las ecuaciones para eliminar una incógnita.

Tipo 11 — Recurrencias Tipo 3: raíz doble

Igual que el Tipo 2, pero el discriminante da 0 y hay una sola raíz repetida.

☑ CÓMO DETECTAR RAÍZ DOBLE

Calcular el discriminante: $b^2 - 4ac$

Si da 0 → raíz doble $r = -b/2a$

Si da positivo → dos raíces distintas (Tipo 2)

☑ ALGORITMO (igual que Tipo 2 salvo la solución general)

1. POLINOMIO CARACTERÍSTICO: igual que Tipo 2

2. RAÍZ DOBLE: $r = -b/2a$ (una sola raíz)

3. SOLUCIÓN GENERAL (DIFERENTE): $xn = (a + b \cdot n) \cdot r^n$

OJO: hay una n multiplicando a b — eso es lo único que cambia

4. SISTEMA: sustituir n=0 y n=1

n=0: $a \cdot r^0 = a = x_0 \rightarrow a = x_0$ directamente

n=1: $(a + b) \cdot r = x_1 \rightarrow$ despejar b

5. Resultado: escribir xn con los valores de a y b

EJEMPLO

$x_0 = 2$, $x_1 = 8$, $xn = 6xn-1 - 9xn-2$

Polinomio: $r^2 - 6r + 9 = 0$

Discriminante: $36 - 36 = 0 \rightarrow$ raíz doble $r = 3$

Solución general: $xn = (a + b \cdot n) \cdot 3^n$

n=0: $a = 2$

n=1: $(2 + b) \cdot 3 = 8 \rightarrow b = 2/3$

Resultado: $xn = (2 + (2/3) \cdot n) \cdot 3^n$

Tipo 12 — Inducción sobre sumas

Demostrar que una fórmula de sumatorio es verdadera para todo n.

ALGORITMO

1. CASO BASE (n=1):

Sustituir n=1 en ambos lados y comprobar que son iguales

2. HIPÓTESIS:

Escribir la fórmula para m (igual que la general pero con m)

3. DESTINO:

Sustituir n por m+1 en la fórmula general — eso es lo que hay que demostrar

4. DEMOSTRACIÓN:

a) Escribir la suma hasta m+1

b) Separar el último término: ∑(hasta m+1) = ∑(hasta m) + último

c) Aplicar la hipótesis al ∑(hasta m)

d) Hacer álgebra hasta llegar al destino

EJEMPLO — $\sum k = n(n+1)/2$

Caso base (n=1): $\sum k = 1 = 1 \cdot 2/2$ ✓

Hipótesis: 1+2+...+m = m(m+1)/2

Destino: 1+2+...+(m+1) = (m+1)(m+2)/2

Dem.: ∑(hasta m+1) = m(m+1)/2 + (m+1) = (m+1)·[m/2+1] = (m+1)(m+2)/2 ✓

⚠ TRAMPAS FRECUENTES

El destino siempre es la fórmula general con n=m+1 sustituido.

El último término se obtiene sustituyendo k=m+1 en el sumando.

Sacar factor común es la clave del álgebra final.

Tipo 13 — Inducción sobre recurrencias

Demostrar que una fórmula xn es solución de una recurrencia dada.

☑ ALGORITMO

1. CASO BASE:
   - Comprobar que la fórmula coincide con los valores iniciales (x_0, x_1)
   - Si la recurrencia usa dos términos → comprobar x_0 Y x_1
2. HIPÓTESIS:
   - Asumir que la fórmula funciona para m−1 (y m−2 si hay dos términos)
3. DESTINO:
   - Demostrar que xm = fórmula(m)
4. DEMOSTRACIÓN:
   - a) Escribir la recurrencia: xm = ...
   - b) Sustituir la hipótesis en xm−1 y xm−2
   - c) Hacer álgebra hasta llegar a xm = fórmula(m)

EJEMPLO — xn = n, con xn = 2xn−1 − xn−2

Caso base: $x_0 = 0$ ✓ y $x_1 = 1$ ✓

Hipótesis: xm−1 = m−1 y xm−2 = m−2

Destino: xm = m

Dem.: xm = 2·(m−1) − (m−2) = 2m−2−m+2 = m ✓

Tipo 13b — Inducción sobre divisibilidad

Demostrar que una expresión es siempre divisible por un número dado.

☑ ALGORITMO (igual que inducción sobre sumas)
1. CASO BASE (n=0 o n=1): calcular la expresión y comprobar que es divisible
2. HIPÓTESIS: asumir que para m, la expresión f(m) es divisible por k
Es decir: f(m) = k·q para algún entero q
3. DESTINO: demostrar que f(m+1) también es divisible por k
4. DEMOSTRACIÓN:
a) Escribir f(m+1) en función de f(m)
b) Separar f(m) (que por hipótesis es múltiplo de k) del resto
c) Demostrar que el resto también es múltiplo de k
d) Concluir que la suma es múltiplo de k

EJEMPLO — $(2n+1)^2-1$ divisible por 8
Caso base (n=0): $(2\cdot0+1)^2-1 = 1-1 = 0$, que es divisible por 8 ✓
Hipótesis: $(2m+1)^2-1 = 8\cdot q$ para algún q
Destino: $(2(m+1)+1)^2-1 = (2m+3)^2-1$ es divisible por 8
Dem.: $(2m+3)^2-1 = (2m+1)^2+8m+8-1 = [(2m+1)^2-1] + 8(m+1)$
$= 8q + 8(m+1) = 8\cdot[q+(m+1)]$ → divisible por 8 ✓

TEMA 4 — Teoría de Grafos

Tipo 16 — Matriz de adyacencia

Obtener la matriz de adyacencia de un grafo, o el grafo a partir de su matriz.

DEFINICIÓN
Matriz cuadrada n×n (n = número de vértices) donde:
A(i,j) = 1 si existe arista entre vi y vj
A(i,j) = 0 si no existe arista entre vi y vj
En grafos no dirigidos: la matriz es SIMÉTRICA (A(i,j) = A(j,i))
En grafos dirigidos: A(i,j) = 1 solo si la arista va de vi a vj

ALGORITMO — del grafo a la matriz
1. Numerar los vértices: v1, v2, ..., vn
2. Crear una tabla n×n con todos los valores a 0
3. Para cada arista {vi, vj}: poner A(i,j) = 1 y A(j,i) = 1
4. La diagonal siempre es 0 (no hay bucles en grafos simples)

ALGORITMO — de la matriz al grafo
1. Cada fila/columna representa un vértice
2. Si A(i,j) = 1 → dibujar arista entre vi y vj
3. Ignorar la diagonal (siempre 0)
4. En grafos no dirigidos, solo mirar el triángulo superior o inferior

Tipo 14 — Caminos y ciclos eulerianos

Determinar si un grafo tiene camino euleriano o ciclo euleriano.

DEFINICIONES
Grado de un vértice: número de aristas que llegan a él
Camino euleriano: pasa por TODAS las aristas exactamente una vez (sin volver al inicio)

Ciclo euleriano: igual que camino pero VUELVE al vértice inicial

☑ ALGORITMO
1. Calcular el grado de CADA vértice (contar sus aristas — las repetidas cuentan doble)
2. Contar cuántos vértices tienen grado IMPAR
3. Aplicar el teorema:
0 impares → CICLO EULERIANO (empieza y termina donde quieras)
2 impares → CAMINO EULERIANO (empieza y termina en los dos impares)
otro número → NI CAMINO NI CICLO
4. Si existe, encontrar el camino/ciclo recorriendo todas las aristas

⚠ TRAMPAS FRECUENTES
Las aristas múltiples (repetidas) cuentan — cada una suma 1 al grado.
No confundir: 2 impares = camino (no ciclo). 0 impares = ciclo.
Para encontrar el camino: empezar en un vértice impar si hay 2.

Tipo 15 — Algoritmo de Dijkstra

Encontrar el camino más corto desde un vértice origen hasta todos los demás.

☑ ALGORITMO
1. Asignar distancia 0 al origen y ∞ al resto
2. Marcar todos como NO visitados
3. BUCLE — repetir hasta visitar todos:
a) Seleccionar el vértice NO visitado con MENOR distancia
b) Para cada vecino no visitado:
nueva_dist = distancia_actual + peso_arista
Si nueva_dist < distancia del vecino → ACTUALIZAR
c) Marcar el vértice como VISITADO
4. Reconstruir el camino: anotar en cada actualización 'viene de qué vértice'
Leer hacia atrás desde el destino para obtener la ruta

⚠ TRAMPAS FRECUENTES
Siempre coger el vértice NO visitado con MENOR distancia — nunca ir directo al destino.
Si la distancia nueva NO mejora la actual → NO actualizar.
El algoritmo calcula distancias a TODOS los vértices, no solo al destino.

EJEMPLO RÁPIDO
Origen v1. Vecinos: v2=3, v3=2 → visitamos v3 (menor).
Desde v3: v2=3 (no mejora), v4=8, v5=5 → actualizamos v4 y v5.
Siguiente menor: v2 (dist=3) → desde v2: v4=5 (mejora de 8 a 5).
Siguiente: v5 (dist=5) → desde v5: v4=6 (no mejora, 5 < 6)... etc.
Al final, leer el camino hacia atrás desde el destino.